FREE GUIDE

# The Verification Engineer Roadmap

## Five steps from curious to hired.

A practical path into chip verification, with one practice project per step and a 12-week plan.

**By Katarine Emanuela Klitzke**
**Former design verification engineer at Microsoft**
**@katarine_emanuela_klitzke**

This is the path I would follow if I were starting again: five steps, one practice project per step, and a 12-week plan to pull it together. I spent 3 years as a design verification engineer at Microsoft, and this is what I wish someone had handed me on day one.

You do not need a perfect background. You need curiosity, patience for debugging, and a habit of asking "what could go wrong?" Work through the steps in order, and do the practice item at the end of each one before you move on.

## What a verification engineer does

Designers build the chip. Verification engineers try to break it before it is manufactured, because a bug found after the chip is made can cost months and a very large amount of money to fix.

On a typical day you will:

- Read a design spec and turn it into a test plan
- Build a testbench that drives the design with realistic and random inputs
- Write checks that flag wrong behavior
- Measure coverage to see what you have not tested yet
- Debug failures in waveforms and work with designers to find the root cause

Verification is often more than half of the effort on a chip project, which is why companies hire so many verification engineers.

## Is this career for you?

Verification can be a great career, and many people love it. It also looks different from the "hacking" image some people have of tech, so it helps to know what the work is really like before you invest time in it.

- **A strong technical base helps.** You need to understand digital design, computer architecture, and the specs well enough to know what the chip should do.
- **There is a lot of reading.** Specs, protocol documents, other people's code, and tool manuals are part of the week.
- **You keep learning.** Languages, methodologies, tools, and designs keep changing.
- **Debugging takes patience.** Sometimes you are close to a bug and still cannot tell where it comes from, and tracking it down can take hours or days. Some people find that satisfying, and others find it frustrating.

It may not be for everyone, and that is completely okay. People enjoy it for different reasons, and your experience may be different from mine. The best way to find out is to try the Step 1 and Step 2 practice projects: they are small, free, and will show you how you feel about the work.

# Step 1: Learn how digital chips think (digital design and RTL)

You cannot verify what you do not understand. Start with how digital hardware works, then learn to describe it in Verilog.

### Learn

- Logic gates, flip-flops, and finite state machines
- Clocks, resets, and why timing matters
- Pipelines, FIFOs, and simple buses
- Verilog basics: modules, always blocks, blocking vs non-blocking assignments

### Practice

1. Write a counter and simulate it
2. Build a FIFO with full and empty flags
3. Build a UART transmitter and receiver
4. Solve exercises on HDLBits, a free browser-based Verilog practice site

### Build it in real life

Simulation is where you will spend most of your time, but building real hardware makes the concepts stick. Try some of these:

- **Prototype on an FPGA board.** Boards such as the Digilent Basys 3, iCEBreaker, or Tang Nano let you run your Verilog on real hardware. Put your counter on LEDs, or connect your UART to a computer terminal.
- **Use a microcontroller to learn the protocols.** An Arduino or ESP32 will not run your Verilog, but you can use one to send and receive UART, SPI, and I2C, and see what those protocols look like on a real wire.
- **Write the protocol yourself.** Read the protocol spec, then implement it from scratch, first in C on a microcontroller and then in Verilog. Do not rely on a ready-made library.
- **Look at the signals.** A cheap logic analyzer or an oscilloscope shows you the real waveforms, and you can compare them with what your simulation predicted.
- **Connect the two.** Have your FPGA design talk to your microcontroller. When something does not match, you get your first taste of debugging across two systems.

**You are done when** you can draw the expected waveform on paper before you run the simulation.

# Step 2: Learn SystemVerilog, the language you use to attack the design

RTL describes the chip. SystemVerilog is how you build the testbench that tests it.

### Learn

- Classes, objects, and interfaces
- Constrained random stimulus: letting the tool generate inputs within rules you set
- Assertions: rules the design must never break
- Functional coverage: a measure of which scenarios you have exercised
- Mailboxes and events for communication between testbench pieces

### Practice

1. Write a class-based testbench for your FIFO from Step 1
2. Drive it with constrained random data and check results against a simple reference model
3. Add three assertions, for example "never write when full"
4. Add coverage bins and push coverage as high as you can

**You are done when** you stop asking "does it work?" and start asking "what did I miss?"

# Step 3: Learn UVM, the industry's testbench framework

UVM (Universal Verification Methodology) is a standard way to structure SystemVerilog testbenches so teams can reuse them. Many job postings ask for it, so it is worth learning, but only after Step 2.

### Learn

- The building blocks: agents, drivers, monitors, sequencers, and scoreboards
- Sequences and sequence items: stimulus you can reuse across tests
- The factory and configuration database
- Phases, and the order in which UVM runs your code

### Practice

1. Rebuild your FIFO testbench from Step 2 as a UVM environment
2. Write two sequences: one directed, one random
3. Add a scoreboard that compares the design output to your reference model
4. Run several tests from one environment

**You are done when** you can explain why each UVM piece exists, not just copy the boilerplate.

# Step 4: Get hands-on with the tools

You can start today without paying for anything. The professional simulators are expensive and mostly learned on the job.

| Tool | What it is for | Cost |
|---|---|---|
| Verilator | Fast open-source simulator for Verilog and SystemVerilog | Free |
| Icarus Verilog | Simple open-source Verilog simulator | Free |
| cocotb | Write testbenches in Python | Free |
| GTKWave | View waveforms | Free |
| EDA Playground | Run simulations in your browser | Free tier |
| Questa, VCS, Xcelium | Commercial simulators used by most chip companies | Paid, learned on the job |

Also build these supporting skills, because they come up every day: Linux command line, Git, and scripting in Python or shell for running regressions and parsing logs.

Note: support for UVM in the free tools is limited, so you may need EDA Playground or a university or employer license for Step 3 practice.

## Step 5: Practice on a real open-source chip

This is the step that turns learning into a portfolio. Real designs come with real testbenches, real bugs, and real documentation.

- **CVA6** is an open-source RISC-V processor core maintained by the OpenHW Group.
- **Ibex** is a small open-source RISC-V core from lowRISC, with a public verification environment.

### Practice

1. Clone the repository and get the build running
2. Read the testbench and draw how its pieces connect
3. Run an existing test and open the waveform
4. Plant a small bug in the design, then write a test that catches it
5. Put your work on GitHub with a short README explaining what you did

**You are done when** you can walk an interviewer through one bug you found and how you found it.

# A 12-week plan

This assumes about 6 to 8 hours a week. Stretch it if you need to; finishing matters more than speed.

| Weeks | Focus | Output |
|---|---|---|
| 1-3 | Step 1: digital design and Verilog | Counter, FIFO, UART |
| 4-6 | Step 2: SystemVerilog | Random testbench with assertions and coverage |
| 7-8 | Step 3: UVM | FIFO testbench rebuilt in UVM |
| 9 | Step 4: tools | Comfortable with Linux, Git, and a simulator |
| 10-12 | Step 5: open-source project | A GitHub repo with a bug you planted and caught |

# Resources to start with

- HDLBits: free Verilog practice in the browser
- *SystemVerilog for Verification* by Chris Spear and Greg Tumbush
- Siemens Verification Academy: free courses and the UVM Cookbook
- The CVA6 and Ibex GitHub repositories

# Where to find designs, simulators, and practice, step by step

| Step | What you need | Where to find it |
|---|---|---|
| 1. Digital design and RTL | Exercises to practice Verilog | HDLBits, ChipVerify tutorials |
| 1. Digital design and RTL | Small open-source designs to read (UART, FIFO, SPI, and more) | OpenCores, GitHub search for "verilog uart" or "verilog fifo" |
| 1. Digital design and RTL | Run your design in the browser, no install | EDA Playground |
| 1. Digital design and RTL | Run it on your own computer | Icarus Verilog, Verilator, GTKWave for waveforms |
| 1. Digital design and RTL | Protocol documents to implement from the spec | I2C: NXP document UM10204. UART and SPI: the datasheet of any microcontroller or chip that uses them |
| 2. SystemVerilog | SystemVerilog examples and tutorials | ChipVerify, Verification Academy |
| 2. SystemVerilog | A simulator that supports classes, randomization, and assertions | EDA Playground (choose a SystemVerilog simulator from its list), or Verilator for part of the language |
| 3. UVM | Courses and the UVM Cookbook | Verification Academy |
| 3. UVM | Run UVM code | EDA Playground (UVM libraries are available there) |
| 4. Tools | Python-based testbenches | cocotb |
| 4. Tools | A bundle of open-source tools in one download | OSS CAD Suite |
| 5. Open-source project | A RISC-V core with a public verification environment | CVA6, Ibex, Core-V-Verif |
| 5. Open-source project | The RISC-V specification, to know what the core should do | riscv.org, under Specifications |

Tip: when you start a project, read its README first. It tells you which tools and versions it expects, and that saves hours of setup trouble.

## Additional skills: Git, project structure, and running tests

Real projects are more than design and testbench files. Learning how they are organized and run will make Step 5 much easier, and it is part of the daily job.

- **Git and GitHub.** Learn to clone, branch, commit, merge, and read a history to see who changed what and why. Free book: Pro Git.
- **Project structure.** Open CVA6 or Ibex and look at how folders are split: design source, testbench, tests, scripts, and docs. Notice where each kind of file lives and how they refer to each other.
- **YAML and config files.** Many projects use YAML or similar files to list tests, choose tool settings, and define pipelines. Learn to read them before you try to write them.
- **Build and run scripts.** Learn the basics of Makefiles and shell scripts, since most projects start a simulation with a make command or a script.
- **Tests and regressions.** A regression is a large set of tests run together, often every night. Learn how a test list is defined, how one test is run on its own, and how to rerun only the failures.
- **Logs and results.** Practice reading simulation logs, finding the first error, and matching it to a point in the waveform. A short Python or shell script that searches logs for "ERROR" saves a lot of time.
- **Continuous integration.** Look at the project's workflow files, for example under `.github/workflows`, to see how tests run automatically on every change. See GitHub Actions documentation.

**Practice:** fork a small open-source design, add your own test to its test list, run it from the command line, and open a pull request or keep it in your own fork with a clear commit message.

## What to do next

Once you finish Step 5, update your resume with the project, link the repo, and apply to design verification and ASIC verification roles. Questions along the way are welcome: send me a DM on Instagram **@katarine_emanuela_klitzke**.